Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий

Кафедра вычислительной техники

ОТЧЁТ О ПРАКТИЧЕСКОЙ РАБОТЕ №3

по дисциплине
«Гибридные вычислительные системы»

«Умножение матриц с использованием разделяемой памяти CUDA»

Преподаватель _____ С. А. Тарасов
подпись, дата инициалы, фамилия

Студент КИ__-__Б,_____ _____ _._. Афонин
номер группы, зачетной книжкой подпись, дата инициалы, фамилия

Студент КИ__-__Б,_____ _____ _._. Никитин
номер группы, зачетной книжкой подпись, дата инициалы, фамилия

Красноярск 2026

СОДЕРЖАНИЕ

1 Цель и задание 3
2 Описание реализации 4
2.1 Структура проекта 4
2.2 Блочный алгоритм умножения матриц 5
2.3 Кёрнел kernel_matmul_shmem 5
2.4 Паттерн «Стратегия» и оператор умножения 7
3 Тестирование 10
4 Методика вычислительного эксперимента 12
5 Результаты измерений 13
6 Анализ результатов 16
6.1 Вычислительная сложность 16
6.2 Ускорение 16
6.3 Сравнение с теоретическими оценками производительности 17
ЗАКЛЮЧЕНИЕ 19
ПРИЛОЖЕНИЕ А (обязательное) Исходный код программы 20

1 Цель и задание

Цель работы: закрепить навыки работы с двумерными сетками нитей CUDA, освоить работу с разделяемой памятью и примитивами синхронизации нитей.

Задание:

– разработать кёрнел `kernel_matmul_shmem`, который принимает объекты `MatrixView` по значению и вычисляет произведение матриц, используя разделяемую память в качестве буфера для обмена данными между нитями; для синхронизации нитей блока использовать `__syncthreads`, не использовать CUDA Cooperative Groups;

– перегрузить оператор `operator*` для класса `Matrix`, используя указанный кёрнел;

– используя Google Test, разработать модульные тесты для `operator*` для матриц A (m×k) и B (k×n), где m, n, k ∈ {1, 2, 3, 127, 128, 129, 512}; эталон – `Eigen::MatrixXf`, сравнение – `Eigen::MatrixXf::isApprox` с точностью 10^{-5};

– используя Google Benchmark и CUDA Events API, разработать бенчмарки для `operator*` для матриц n×n, n ∈ {16, 32, 64, 128, 256, 512, 1024}, без учёта времени выделения, копирования и освобождения памяти;

– построить графики реальной вычислительной сложности умножения матриц для новой и предыдущей версий `operator*`, а также график ускорения новой версии относительно предыдущей;

– сравнить экспериментальные результаты с теоретическими оценками вычислительной сложности.

2 Описание реализации

2.1 Структура проекта

Проект является развитием практической работы №2 и собирается с помощью CMake и Ninja, используется стандарт C++20 и CUDA 13.3. Зависимости (Google Test 1.14, Google Benchmark 1.8.3, Eigen 3.4.0) подключаются через механизм FetchContent. Проект состоит из трёх подпроектов:

– `core` – библиотека с классами `Data`, `MatrixView`, `Matrix`, стратегиями умножения и кёрнелами;

– `tests` – модульные тесты на Google Test;

– `benchmarks` – бенчмарки на Google Benchmark.

В каталоге `scripts` находятся скрипты на Python (пакет plotly) для построения графиков, в каталоге `results` – результаты измерений в формате JSON. Полная реализация доступна в репозитории https://github.com/nameofthelove/gvs-afonin-nikitin (ветка work3). Ключевые фрагменты кода приведены в тексте отчёта, полный исходный код – в приложении А.

Классы `Data` (владение памятью устройства по идиоме RAII) и `MatrixView` (тривиально копируемое невладеющее представление матрицы в порядке row-major) перенесены из работы №2 без изменений. Новые и изменённые компоненты библиотеки приведены в таблице 1.

Таблица 1 – Новые и изменённые компоненты библиотеки core

Компонент	Назначение
`kernel_matmul_shmem`	Блочное умножение матриц с буферизацией тайлов в разделяемой памяти (kernel.cuh)
`launch_matmul_shmem`	Расчёт конфигурации сетки и асинхронный запуск кёрнела, проверка ошибки запуска
`MatmulStrategy<AtomT>`	Абстрактная стратегия умножения: метод multiply(a, b, c, stream)
`NaiveMatmul<AtomT>`	Стратегия на основе kernel_matmul_naive (предыдущая версия operator*)
`SharedMemoryMatmul<AtomT, TileDim>`	Стратегия на основе kernel_matmul_shmem с тайлом TileDim×TileDim, по умолчанию TileDim = 16

Компонент	Назначение
`Matrix<AtomT>`	Фасад: хранит стратегию в std::shared_ptr, operator* делегирует ей вычисление

2.2 Блочный алгоритм умножения матриц

В наивном алгоритме каждая нить вычисляет один элемент $C(i, j) = \Sigma A(i, p) \cdot B(p, j)$, читая из глобальной памяти строку A и столбец B. Нити одного блока при этом многократно читают одни и те же элементы: всего выполняется $2 \cdot m \cdot n \cdot k$ обращений к глобальной памяти, т. е. одна загрузка на каждую операцию с плавающей точкой.

В блочном алгоритме матрица C разбивается на квадратные тайлы T×T, каждый тайл вычисляется одним блоком из T×T нитей. Внутренняя размерность k проходится полосами ширины T: на каждом шаге блок загружает в разделяемую память тайл A размером T×T (из строк тайла C) и тайл B того же размера (из столбцов тайла C), каждая нить загружает по одному элементу каждого тайла. После синхронизации каждая нить накапливает скалярное произведение строки тайла A на столбец тайла B, читая данные из разделяемой памяти. Каждый загруженный элемент используется T нитями, поэтому число обращений к глобальной памяти уменьшается в T раз: до $2 \cdot m \cdot n \cdot k / T$.

2.3 Кёрнел kernel_matmul_shmem

Реализация кёрнела приведена в листинге 1. Кёрнел является шаблоном по типу элементов `AtomT` и размеру тайла `TileDim`; тайлы объявлены как статические массивы `__shared__ AtomT tile_a[TileDim][TileDim]`,

что позволяет компилятору знать их размер и полностью развернуть внутренний цикл (`#pragma unroll`).

Листинг 1 – Кёрнел kernel_matmul_shmem (core/include/kernel.cuh)

```
template <typename AtomT, unsigned int TileDim>
__global__ void kernel_matmul_shmem(MatrixView<AtomT> a, MatrixView<AtomT> b,
                                    MatrixView<AtomT> c) {
    __shared__ AtomT tile_a[TileDim][TileDim];
    __shared__ AtomT tile_b[TileDim][TileDim];

    const unsigned int tx = threadIdx.x;
    const unsigned int ty = threadIdx.y;
    const std::size_t row = static_cast<std::size_t>(blockIdx.y) * TileDim + ty;
    const std::size_t col = static_cast<std::size_t>(blockIdx.x) * TileDim + tx;

    AtomT acc{};
    for (std::size_t tile_start = 0; tile_start < a.ncols(); tile_start += TileDim) {
        // Соседние по tx нити читают соседние элементы строки — доступ объединяется.
        const std::size_t a_col = tile_start + tx;
        const std::size_t b_row = tile_start + ty;
        tile_a[ty][tx] = (row < a.nrows() && a_col < a.ncols()) ? a(row, a_col) :
AtomT{};
        tile_b[ty][tx] = (b_row < b.nrows() && col < b.ncols()) ? b(b_row, col) :
AtomT{};

        // Тайлы должны быть полностью загружены до начала вычислений.
        __syncthreads();

#pragma unroll
        for (unsigned int p = 0; p < TileDim; ++p) {
            // tile_a[ty][p] одинаков для всего варпа (broadcast),
            // tile_b[p][tx] — последовательные слова, конфликтов банков нет.
            acc += tile_a[ty][p] * tile_b[p][tx];
        }

        // Тайлы нельзя перезаписывать, пока все нити блока не закончат их чтение.
        __syncthreads();
    }

    // Проверка выполняется после цикла: все нити блока, включая лежащие за границами
C,
    // должны участвовать в загрузке тайлов и доходить до __syncthreads.
    if (row < c.nrows() && col < c.ncols()) {
        c(row, col) = acc;
    }
}
```

Ключевые решения:

– используется двумерная сетка блоков TileDim×TileDim: ось x соответствует столбцам C, ось y – строкам; размер сетки вычисляется с округлением вверх: gridDim.x = ⌈n / T⌉, gridDim.y = ⌈m / T⌉;

– при загрузке тайлов соседние по threadIdx.x нити читают соседние элементы строки матрицы, поэтому обращения варпа к глобальной памяти объединяются (coalesced access);

– элементы, выходящие за границы матриц A и B, заменяются нулями; это позволяет обрабатывать размеры, не кратные T, без ветвлений во внутреннем цикле;

– первый вызов `__syncthreads()` гарантирует, что тайлы полностью загружены до начала вычислений, второй – что ни одна нить не начнёт перезаписывать тайлы следующей полосы, пока остальные нити блока их читают;

– проверка row < m и col < n выполняется только при записи результата: все нити блока, в том числе вышедшие за границы C, обязаны участвовать в загрузке тайлов и доходить до `__syncthreads()`, иначе поведение программы не определено;

– во внутреннем цикле элемент tile_a[ty][p] одинаков для всех нитей варпа (широковещательное чтение), а элементы tile_b[p][tx] лежат в последовательных словах и попадают в разные банки разделяемой памяти, поэтому конфликты банков отсутствуют.

Размер тайла по умолчанию выбран равным 16: блок 16×16 = 256 нитей использует 2 КБ разделяемой памяти, и на мультипроцессоре архитектуры Ampere (до 1536 нитей) одновременно размещается 6 блоков, т. е. достигается 100 % теоретической загрузки. При тайле 32×32 (1024 нити) на мультипроцессор помещается только один блок, загрузка снижается до 67 %, а при синхронизации блока мультипроцессору нечем скрыть задержки. Это подтверждается измерениями (раздел 5).

2.4 Паттерн «Стратегия» и оператор умножения

Алгоритм умножения вынесен в иерархию стратегий (листинг 2). Абстрактный класс `MatmulStrategy` объявляет метод `multiply`, принимающий уже согласованные по размерам представления матриц; конкретные стратегии `NaiveMatmul` и `SharedMemoryMatmul` вызывают соответствующие функции запуска кёрнелов. Класс `Matrix` хранит стратегию в `std::shared_ptr<const MatmulStrategy>`: копии матриц разделяют

один объект стратегии, а по умолчанию используется общий экземпляр `SharedMemoryMatmul`. Стратегию можно передать в конструктор или заменить методом `set_matmul_strategy`; нулевой указатель отвергается исключением std::invalid_argument.

Листинг 2 – Стратегии умножения (core/include/matmul_strategy.cuh)

```
template <typename AtomT>
class MatmulStrategy {
public:
    virtual ~MatmulStrategy() = default;

    virtual void multiply(MatrixView<AtomT> a, MatrixView<AtomT> b, MatrixView<AtomT>
c,
                          cudaStream_t stream = nullptr) const = 0;
};

// Наивный алгоритм: каждая нить читает строку A и столбец B из глобальной памяти.
template <typename AtomT>
class NaiveMatmul final : public MatmulStrategy<AtomT> {
public:
    void multiply(MatrixView<AtomT> a, MatrixView<AtomT> b, MatrixView<AtomT> c,
                  cudaStream_t stream = nullptr) const override {
        launch_matmul_naive(a, b, c, stream);
    }
};

// Блочный алгоритм с буферизацией тайлов TileDim x TileDim в разделяемой памяти.
template <typename AtomT, unsigned int TileDim = kMatmulTileDim>
class SharedMemoryMatmul final : public MatmulStrategy<AtomT> {
public:
    void multiply(MatrixView<AtomT> a, MatrixView<AtomT> b, MatrixView<AtomT> c,
                  cudaStream_t stream = nullptr) const override {
        launch_matmul_shmem<AtomT, TileDim>(a, b, c, stream);
    }
};
```

Оператор `operator*` (листинг 3) проверяет согласованность размеров, создаёт матрицу результата с той же стратегией, что и у левого операнда, делегирует вычисление стратегии и дожидается завершения кёрнела вызовом cudaDeviceSynchronize, преобразуя возможную ошибку выполнения в исключение. Такое разделение позволяет тестировать и измерять разные алгоритмы через единый интерфейс, не изменяя класс Matrix.

Листинг 3 – Оператор умножения (core/include/matrix.cuh)

```
    // C = (*this) * other, где (*this) — m x k, other — k x n, C — m x n.
    // Умножение выполняет стратегия левого операнда; результат наследует её.
    Matrix operator*(const Matrix& other) const {
        if (ncols() != other.nrows()) {
            throw std::invalid_argument("Matrix dimensions mismatch: lhs.ncols != 
rhs.nrows");
        }

        Matrix result(nrows(), other.ncols(), matmul_strategy_);
        matmul_strategy_->multiply(view_, other.view_, result.view_);
        check_cuda(cudaDeviceSynchronize(), "matmul execution");
        return result;
    }
```

3 Тестирование

Модульные тесты разработаны с использованием Google Test (листинг А.12). В каждом параметризованном тесте генерируются случайные матрицы A (m×k) и B (k×n), вычисляется эталонное произведение средствами Eigen на CPU и произведение `operator*` на GPU, после чего проверяются размеры результата и выполняется сравнение методом `Eigen::MatrixXf::isApprox` с точностью 10^{-5}. Обмен данными с Eigen выполняется через промежуточную матрицу с порядком хранения RowMajor (листинг А.11). Наборы тестов приведены в таблице 2.

Таблица 2 – Наборы модульных тестов

Набор	Число тестов	Что проверяется
AllSizes/MatmulParamTest	$7^3 = 343$	operator* со стратегией по умолчанию (kernel_matmul_shmem) для всех m, k, n
AllStrategies/ MatmulStrategyParamTest	$3 \cdot 7^3 = 1029$	Те же размеры для стратегий NaiveMatmul, SharedMemoryMatmul<16> и SharedMemoryMatmul<32>
MatmulTest, MatmulStrategyTest, MatrixTest, MatrixViewTest, DataTest	21	Граничные случаи, свойства классов и стратегий, отсутствие утечек памяти

Выбранные размеры покрывают характерные для блочного алгоритма ситуации: 1, 2, 3 – матрица меньше одного тайла, большая часть нитей блока загружает нули; 128 – размер кратен 16 и 32, все тайлы заполнены полностью; 127 и 129 – последний тайл заполнен частично (на одну строку/столбец меньше или больше); 512 – большое число полос по внутренней размерности. Комбинации с k = 1 проверяют случай одной неполной полосы. Дополнительные тесты приведены в таблице 3.

Таблица 3 – Дополнительные тесты

Тест	Что проверяется
KnownSmallProduct	Произведение 2×3 · 3×2 с заранее известным результатом
IdentityIsNeutral	A·I = I·A = A для матрицы 129×129
VectorShapes	Строка 1×1000 на столбец 1000×1 и наоборот: много полос, вырожденная сетка

Тест	Что проверяется
TileBoundaries	Размеры 15, 16, 17, 31, 32, 33, 63, 65 вокруг границ тайлов
RepeatedProductIsStable	Повторное умножение даёт тот же результат
OperandsAreNotModified	Операнды не изменяются, результат в новом буфере
DimensionMismatchThrows	Несогласованные размеры – std::invalid_argument
ZeroInnerDimension GivesZeroMatrix	k = 0: результат m×n заполнен нулями
EmptyResult	m = 0: кёрнел не запускается, ошибок CUDA нет
DoublePrecision, IntegerProductIsExact	Шаблон работает для double и int
NoDeviceMemoryLeak	После 50 умножений 512×512 с копированием объём свободной памяти GPU (cudaMemGetInfo) не изменился
DefaultIsSharedMemory	Стратегия по умолчанию – SharedMemoryMatmul
ResultInheritsLhsStrategy	Результат умножения использует стратегию левого операнда
NullStrategyThrows	Нулевая стратегия отвергается
StrategiesAgree	Наивная и блочная стратегии дают совпадающий результат
MatrixViewTest, MatrixTest, DataTest	Тривиальная копируемость MatrixView, row-major индексация, разделение данных и стратегии копиями Matrix, глубокое копирование и перемещение Data

Все 1393 теста выполнены успешно (Windows 11, MSVC 19.42, CUDA 13.3). Отсутствие утечек обеспечивается владением памятью через RAII-классы Data и CudaEventTimer и проверяется тестом NoDeviceMemoryLeak; все вызовы CUDA Runtime API проверяются функцией check_cuda.

4 Методика вычислительного эксперимента

Измерения выполнены на следующей конфигурации: GPU NVIDIA GeForce RTX 3050 Ti Laptop GPU (архитектура Ampere, compute capability 8.6, 20 SM, 2560 ядер CUDA, частота до 1485 МГц, 4 ГБ GDDR6 с шиной 128 бит, кэш L2 2 МБ, разделяемая память до 48 КБ на блок), CPU Intel Core i7-12650H, ОС Windows 11, драйвер 616.92, CUDA 13.3, MSVC 19.42, сборка Release, архитектура sm_86. Параметры GPU определяются программой бенчмарков через cudaGetDeviceProperties и сохраняются в поле context JSON-отчёта.

Бенчмарки разработаны с использованием Google Benchmark (листинг А.15) для квадратных матриц n×n, n ∈ {16, 32, 64, 128, 256, 512, 1024}. Шаблонная функция BM_Matrix_operator_mul параметризуется стратегией, что позволяет измерять предыдущую (NaiveMatmul) и новую (SharedMemoryMatmul с тайлами 16×16 и 32×32) реализации одним и тем же кодом:

– матрицы A, B и C создаются и заполняются случайными значениями до цикла измерений, поэтому выделение, копирование и освобождение памяти в замер не входят;

– перед циклом выполняется прогревочный запуск (загрузка модуля, выход GPU из режима энергосбережения);

– в цикле измеряется только время выполнения кёрнела – того же вызова strategy.multiply, который выполняет operator*, – с помощью пары CUDA Events (RAII-обёртка CudaEventTimer, листинг А.14); время передаётся в Google Benchmark методом SetIterationTime (режим UseManualTime);

– каждый бенчмарк повторяется 10 раз, в качестве результата используется медиана, на графиках также показано стандартное отклонение.

5 Результаты измерений

Результаты измерений приведены в таблице 4. Производительность вычислена исходя из $2n^3$ операций с плавающей точкой на одно умножение, ускорение – как S(n) = T_naive(n) / T_shmem(n).

Таблица 4 – Время умножения матриц n×n (медиана, мкс), производительность (GFLOP/s) и ускорение

n	T naive	P naive	T shmem 16	P shmem 16	T shmem 32	P shmem 32	S 16	S 32
16	14,0	0,6	14,1	0,6	16,5	0,5	0,99	0,85
32	15,5	4,2	16,4	4,0	18,4	3,6	0,94	0,84
64	14,9	35,3	15,4	34,1	16,5	31,8	0,97	0,90
128	19,2	217,9	15,9	263,9	15,8	264,9	1,21	1,22
256	74,5	450,2	54,1	620,3	64,7	519,0	1,38	1,15
512	543,0	494,3	407,3	659,0	428,9	625,8	1,33	1,27
1024	4583,3	468,5	3565,4	602,3	3766,6	570,1	1,29	1,22

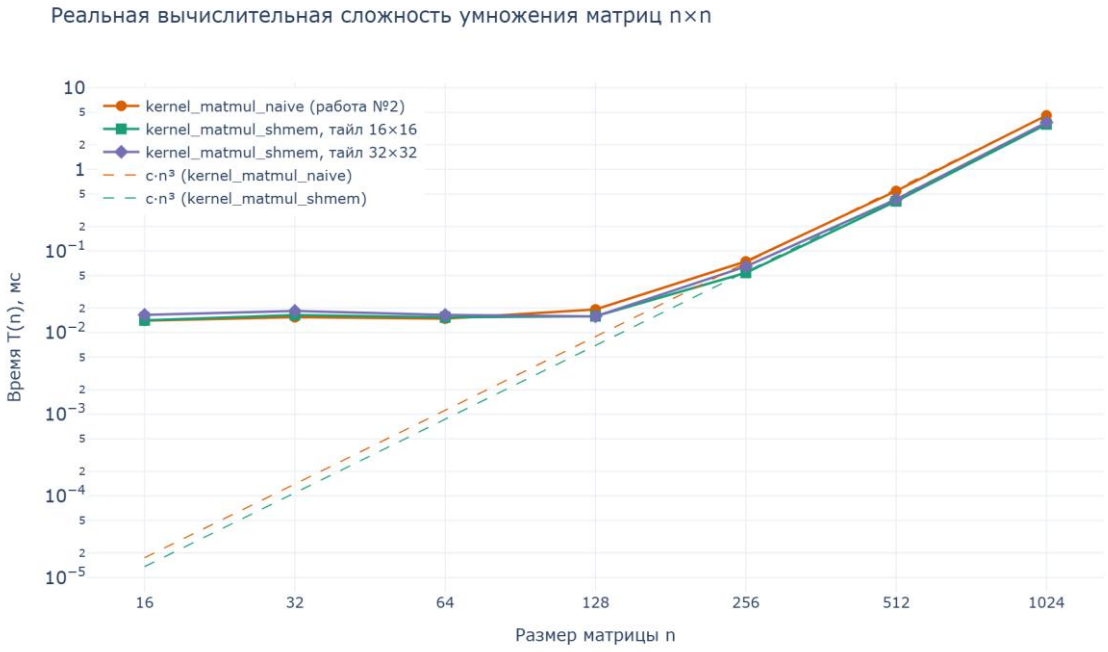


Рисунок 1 – Графики реальной вычислительной сложности T(n) (пунктир – теоретическая зависимость $c \cdot n^3$)

На рисунке 1 изображены графики реальной вычислительной сложности умножения матриц для предыдущей (kernel_matmul_naive) и новой (kernel_matmul_shmem) реализаций operator* в логарифмическом масштабе.

Пунктирные линии соответствуют теоретической зависимости $c \cdot n^3$, проведённой через последнюю точку серии.

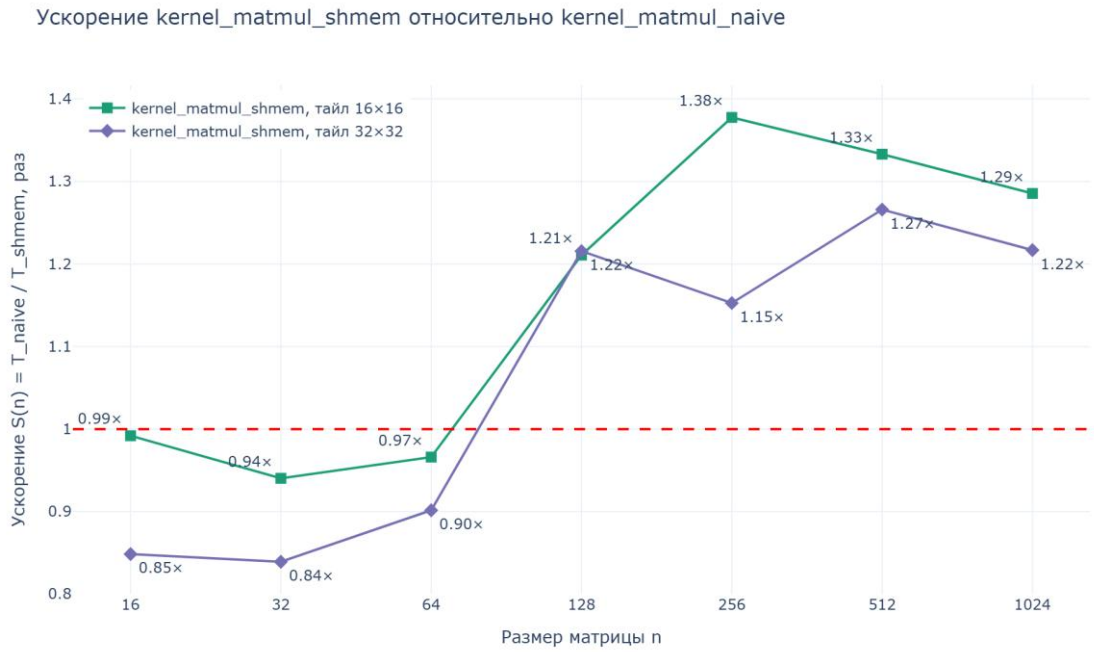


Рисунок 2 – График ускорения S(n) = T_naive(n) / T_shmem(n)

На рисунке 2 изображён график ускорения новой реализации относительно предыдущей для тайлов 16×16 и 32×32. Пунктирная линия S = 1 соответствует равной производительности.

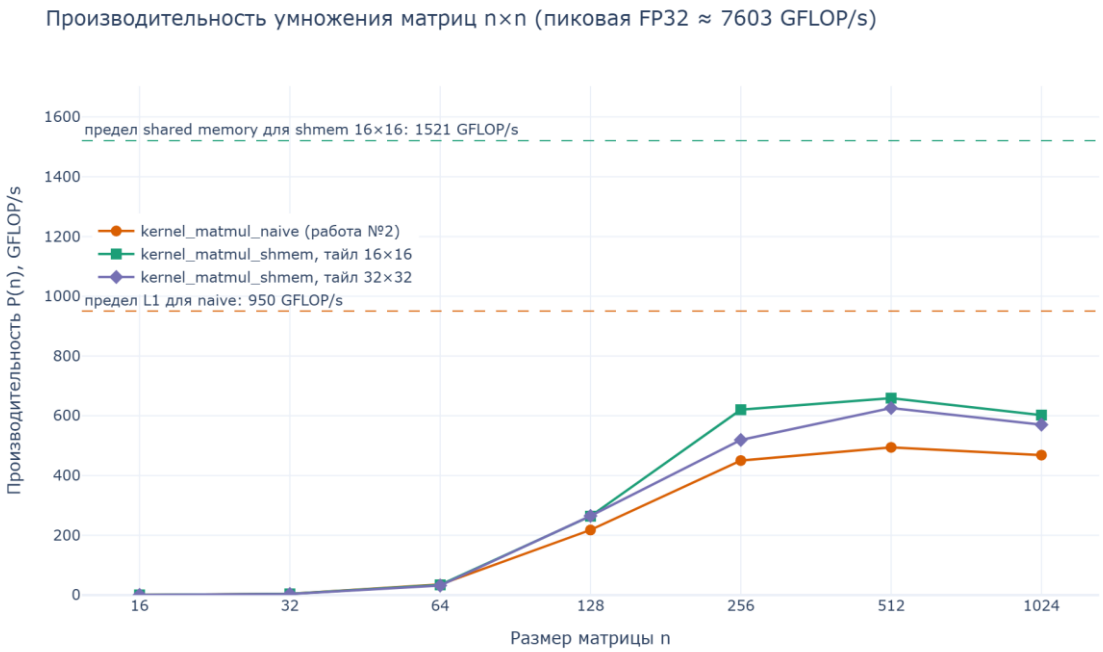


Рисунок 3 – Производительность P(n) и теоретические верхние оценки для каждого кёрнела

На рисунке 3 изображена достигнутая производительность; пунктирные линии – теоретические верхние оценки производительности кёрнелов, полученные в разделе 6.

6 Анализ результатов

6.1 Вычислительная сложность

Умножение матриц A (m×k) и B (k×n) требует m·n·k умножений и столько же сложений, т. е. имеет вычислительную сложность O(m·n·k), для квадратных матриц – $O(n^3)$. Блочный алгоритм не изменяет число арифметических операций, а лишь уменьшает число обращений к глобальной памяти в T раз, поэтому обе реализации имеют сложность $O(n^3)$ и должны различаться только константой c. В логарифмических координатах зависимость $T(n) = c \cdot n^p$ является прямой с наклоном p = 3.

Показатель степени p оценён методом наименьших квадратов по логарифмам измеренных значений (таблица 5).

Таблица 5 – Эмпирический показатель степени p в модели $T(n) = c \cdot n^p$

Реализация	p (все n)	p (n ≥ 256)
kernel_matmul_naive	1,34	2,97
kernel_matmul_shmem, тайл 16×16	1,25	3,02
kernel_matmul_shmem, тайл 32×32	1,23	2,93

При n ≥ 256 показатель для всех реализаций близок к трём (2,93–3,02), что соответствует теоретической оценке $O(n^3)$. При n ≤ 128 время практически не зависит от размера и составляет 14,0–18,4 мкс: оно определяется фиксированными накладными расходами на запуск кёрнела и запись событий в драйвере WDDM, а также неполной загрузкой GPU – при n = 64 и тайле 16 запускается всего 16 блоков на 20 мультипроцессоров. Поэтому оценка p по всем точкам (1,2–1,3) не отражает асимптотику.

6.2 Ускорение

При n ≤ 64 блочный кёрнел не даёт выигрыша (S = 0,94–0,99 для тайла 16 и 0,84–0,90 для тайла 32): время определяется накладными расходами на запуск, а дополнительные синхронизации и загрузка тайлов, частично заполненных нулями, лишь добавляют работу. Начиная с n = 128 новая реализация быстрее предыдущей; максимальное ускорение S = 1,38

достигнуто при n = 256, при n = 512 и n = 1024 ускорение составляет 1,33 и 1,29.

Тайл 32×32 во всём диапазоне уступает тайлу 16×16 из-за более низкой загрузки мультипроцессоров (раздел 2.3). Особенно заметен провал при n = 256 (S = 1,15): сетка из 8×8 = 64 блоков по 1024 нити выполняется на 20 мультипроцессорах в 4 волны, последняя из которых загружает только 4 SM, тогда как при тайле 16 256 блоков распределяются по 120 слотам (6 блоков × 20 SM) равномернее.

6.3 Сравнение с теоретическими оценками производительности

Для объяснения величины ускорения оценим верхние границы производительности кёрнелов (таблица 6). Пиковая производительность GPU в FP32 составляет 2·128·20·1,485 ГГц ≈ 7,6 TFLOP/s, пропускная способность памяти – 6001 МГц·2·128 бит / 8 ≈ 192 ГБ/с.

По дизассемблированному коду (cuobjdump -sass) внутренний цикл kernel_matmul_shmem для тайла 16 содержит 16 инструкций FFMA, 16 инструкций LDS (чтение tile_b) и 4 инструкции LDS.128 (компилятор объединил чтение четырёх соседних элементов строки tile_a в одну векторную инструкцию). Разделяемая память мультипроцессора выдаёт 128 байт за такт, т. е. обслуживает одну инструкцию LDS варпа за такт; следовательно, на 16 инструкций FFMA варпа требуется не менее 20 тактов, и производительность ограничена величиной 2·32·(16/20)·20·1,485 ГГц ≈ 1521 GFLOP/s. Наивный кёрнел на каждую FFMA выполняет две загрузки из глобальной памяти, которые в лучшем случае обслуживаются кэшем L1 с той же пропускной способностью 128 байт/такт, что даёт предел ≈ 950 GFLOP/s.

Таблица 6 – Теоретические оценки и достигнутые значения (n = 512)

Показатель	**naive**	**shmem 16×16**
Обращений к глобальной памяти, элементов	$2n^3$	$2n^3 / 16$
Объём чтения из глобальной памяти при n = 1024 (без учёта кэшей)	8,6 ГБ	0,54 ГБ
Время чтения этого объёма из DRAM при n = 1024	44,7 мс	2,8 мс

Показатель	naive	shmem 16×16
Верхняя оценка производительности, GFLOP/s	950	1521
Достигнутая производительность, GFLOP/s	494	659
Доля от верхней оценки	52 %	43 %
Доля от пиковой производительности FP32	6,5 %	8,7 %

Если бы наивный кёрнел читал все данные из DRAM, умножение матриц 1024×1024 заняло бы не менее 45 мс, фактически оно длится 4,6 мс. Это означает, что на архитектуре Ampere повторное использование данных в наивном кёрнеле уже обеспечивается аппаратно кэшами L1 и L2, причём кэш L1 и разделяемая память физически являются одним и тем же блоком памяти мультипроцессора (128 КБ на SM) с одинаковой пропускной способностью. Поэтому явная буферизация в разделяемой памяти даёт выигрыш не за счёт более быстрой памяти, а за счёт сокращения числа инструкций загрузки на одну FFMA (с 2 до 1,25), гарантированного повторного использования данных без промахов кэша и векторизации чтения. Отношение верхних оценок 1521 / 950 ≈ 1,60 ограничивает достижимое ускорение; измеренное ускорение 1,29–1,38 составляет 80–85 % от этой величины, что согласуется с теоретической оценкой.

Обе реализации достигают 40–55 % от своих верхних оценок и менее 10 % от пиковой производительности GPU. Оставшиеся потери связаны с барьерной синхронизацией (пока часть варпов блока ждёт на `__syncthreads()`, они не выполняют вычислений), с задержками загрузки тайлов из глобальной памяти, не перекрытыми вычислениями, и с накладными расходами на 64-битную адресную арифметику. Снижение производительности при переходе от n = 512 к n = 1024 объясняется тем, что матрицы размером 4 МБ перестают помещаться в кэш L2 объёмом 2 МБ. Дальнейшее повышение производительности возможно за счёт вычисления каждой нитью нескольких элементов C (register tiling), что увеличивает число FFMA на одну загрузку из разделяемой памяти, и за счёт двойной буферизации тайлов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы закреплены навыки работы с двумерными сетками нитей CUDA, освоены работа с разделяемой памятью и синхронизация нитей блока с помощью `__syncthreads`.

Разработан кёрнел kernel_matmul_shmem, реализующий блочный алгоритм умножения матриц с буферизацией тайлов в разделяемой памяти и поддержкой размеров, не кратных размеру тайла. Оператор Matrix::operator* переработан по паттерну «Стратегия»: алгоритм умножения инкапсулирован в классах NaiveMatmul и SharedMemoryMatmul, по умолчанию используется блочный алгоритм с тайлом 16×16. Корректность реализации подтверждена 1393 модульными тестами с эталоном Eigen, включая все комбинации размеров из задания для каждой стратегии, граничные случаи и проверку отсутствия утечек памяти устройства.

Экспериментально подтверждена кубическая вычислительная сложность умножения матриц для обеих реализаций ($p \approx 2{,}97$ и $p \approx 3{,}02$ при $n \geq 256$). Новая реализация быстрее предыдущей начиная с $n = 128$, максимальное ускорение составило 1,38 раза при $n = 256$, максимальная производительность – 659 GFLOP/s. Умеренная величина ускорения согласуется с теоретической оценкой: на архитектуре Ampere наивный кёрнел уже эффективно использует кэш L1, а блочный кёрнел ограничен пропускной способностью разделяемой памяти, что даёт предельное ускорение около 1,6 раза.

ПРИЛОЖЕНИЕ А

(обязательное)

Исходный код программы

Листинг А.1 – Файл `CMakeLists.txt`

```
cmake_minimum_required(VERSION 3.24)

if(POLICY CMP0135)
  cmake_policy(SET CMP0135 NEW)
endif()

project(work3 LANGUAGES CXX CUDA)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CUDA_STANDARD 20)
set(CMAKE_CUDA_STANDARD_REQUIRED ON)

if(NOT CMAKE_BUILD_TYPE AND NOT CMAKE_CONFIGURATION_TYPES)
  set(CMAKE_BUILD_TYPE Release CACHE STRING "Build type" FORCE)
endif()

# Архитектура GPU (Ampere: RTX 3050 Ti -> sm_86); переопределяется через -
     DCMAKE_CUDA_ARCHITECTURES
if(NOT DEFINED CMAKE_CUDA_ARCHITECTURES)
  set(CMAKE_CUDA_ARCHITECTURES 86)
endif()

if(MSVC)
  # Исходники в UTF-8 (комментарии на русском); C4068 — прагмы nvcc в заголовках Eigen
  add_compile_options(
    $<$<COMPILE_LANGUAGE:CXX>:/utf-8>
    $<$<COMPILE_LANGUAGE:CUDA>:-Xcompiler=/utf-8,/wd4068>
  )
endif()

find_package(CUDAToolkit REQUIRED)

include(FetchContent)

# Google Test
FetchContent_Declare(
  googletest
  URL https://github.com/google/googletest/archive/refs/tags/v1.14.0.zip
)
set(gtest_force_shared_crt ON CACHE BOOL "" FORCE)
set(INSTALL_GTEST OFF CACHE BOOL "" FORCE)
FetchContent_MakeAvailable(googletest)

# Google Benchmark
FetchContent_Declare(
  googlebenchmark
  URL https://github.com/google/benchmark/archive/refs/tags/v1.8.3.zip
)
set(BENCHMARK_ENABLE_TESTING OFF CACHE BOOL "" FORCE)
set(BENCHMARK_ENABLE_INSTALL OFF CACHE BOOL "" FORCE)
FetchContent_MakeAvailable(googlebenchmark)
```

```
# Eigen 3
FetchContent_Declare(
  eigen
  GIT_REPOSITORY https://gitlab.com/libeigen/eigen.git
  GIT_TAG 3.4.0
  GIT_SHALLOW TRUE
)
set(EIGEN_BUILD_DOC OFF CACHE BOOL "" FORCE)
set(EIGEN_BUILD_TESTING OFF CACHE BOOL "" FORCE)
set(EIGEN_BUILD_PKGCONFIG OFF CACHE BOOL "" FORCE)
set(BUILD_TESTING OFF CACHE BOOL "" FORCE)
FetchContent_MakeAvailable(eigen)


enable_testing()


add_subdirectory(core)
add_subdirectory(tests)
add_subdirectory(benchmarks)
```

Листинг А.2 – Файл `core/CMakeLists.txt`

```
add_library(core STATIC
    src/matrix.cu
)


target_include_directories(core PUBLIC
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)


target_link_libraries(core PUBLIC
    CUDA::cudart
)
```

Листинг А.3 – Файл `core/include/cuda_check.cuh`

```
#pragma once


#include <cuda_runtime.h>
#include <stdexcept>
#include <string>


// Преобразует код ошибки CUDA Runtime API в исключение std::runtime_error.
inline void check_cuda(cudaError_t err, const char* what) {
    if (err != cudaSuccess) {
        throw std::runtime_error(std::string(what) + ": " + cudaGetErrorString(err));
    }
}
```

Листинг А.4 – Файл `core/include/data.cuh`

```
#pragma once


#include <cstddef>
#include <cuda_runtime.h>
#include <utility>


#include "cuda_check.cuh"


// Владеющий RAII-буфер в глобальной памяти устройства.
template <typename AtomT>
```

```
class Data {
private:
    std::size_t size_;
    AtomT* data_;


public:
    explicit Data(std::size_t size) : size_(size), data_(nullptr) {
        if (size_ > 0) {
            check_cuda(cudaMalloc(&data_, size_ * sizeof(AtomT)), "cudaMalloc");
        }
    }


    ~Data() {
        if (data_ != nullptr) {
            cudaFree(data_);
        }
    }


    Data(const Data& other) : Data(other.size_) { //cop
        if (size_ > 0) {
            // При исключении память освободит деструктор делегирующего конструктора.
            check_cuda(cudaMemcpy(data_, other.data_, size_ * sizeof(AtomT),
                                  cudaMemcpyDeviceToDevice),
                       "cudaMemcpy (D2D)");
        }
    }


    Data(Data&& other) noexcept
        : size_(std::exchange(other.size_, 0)), data_(std::exchange(other.data_,
     nullptr)) {}


    Data& operator=(const Data& other) {
        if (this != &other) {
            Data temp(other);
            std::swap(size_, temp.size_);
            std::swap(data_, temp.data_);
        }
        return *this;
    }


    Data& operator=(Data&& other) noexcept {
        if (this != &other) {
            if (data_ != nullptr) {
                cudaFree(data_);
            }
            size_ = std::exchange(other.size_, 0);
            data_ = std::exchange(other.data_, nullptr);
        }
        return *this;
    }


    void copy_to_host(AtomT* host_ptr) const {
        if (size_ == 0) {
            return;
        }
        check_cuda(cudaMemcpy(host_ptr, data_, size_ * sizeof(AtomT),
     cudaMemcpyDeviceToHost),
                   "cudaMemcpy (D2H)");
    }


    void copy_from_host(const AtomT* host_ptr) {
        if (size_ == 0) {
            return;
        }
        check_cuda(cudaMemcpy(data_, host_ptr, size_ * sizeof(AtomT),
```

```
     cudaMemcpyHostToDevice),
                  "cudaMemcpy (H2D)");
    }


    AtomT* data() { return data_; }
    const AtomT* data() const { return data_; }
    std::size_t size() const { return size_; }
};
```

Листинг А.5 – Файл `core/include/matrix_view.cuh`

```
#pragma once


#include <cstddef>
#include <cuda_runtime.h>
#include <type_traits>


// Невладеющее представление матрицы nrows x ncols в линейной памяти (row-major).
// Тривиально копируемо, поэтому передаётся в кёрнелы по значению.
template <typename AtomT>
class MatrixView {
private:
    AtomT* data_;
    std::size_t nrows_;
    std::size_t ncols_;


public:
    __host__ __device__ MatrixView(AtomT* data, std::size_t nrows, std::size_t ncols)
        : data_(data), nrows_(nrows), ncols_(ncols) {}


    __host__ __device__ std::size_t size() const { return nrows_ * ncols_; }
    __host__ __device__ std::size_t nrows() const { return nrows_; }
    __host__ __device__ std::size_t ncols() const { return ncols_; }


    // Доступ по линейному индексу.
    __host__ __device__ AtomT& operator[](std::size_t n) { return data_[n]; }
    __host__ __device__ const AtomT& operator[](std::size_t n) const { return
     data_[n]; }


    // Доступ по (строка, столбец): элемент (i, j) лежит по смещению i * ncols + j.
    __host__ __device__ AtomT& operator()(std::size_t i, std::size_t j) {
        return data_[i * ncols_ + j];
    }
    __host__ __device__ const AtomT& operator()(std::size_t i, std::size_t j) const {
        return data_[i * ncols_ + j];
    }
};


static_assert(std::is_trivially_copyable_v<MatrixView<float>>,
              "MatrixView must be trivially copyable to be passed to kernels by
     value");
```

Листинг А.6 – Файл `core/include/kernel.cuh`

```
#pragma once


#include <cstddef>
#include <cuda_runtime.h>


#include "cuda_check.cuh"
#include "matrix_view.cuh"
```

```
// Сторона квадратного блока нитей для наивного кёрнела (16 x 16 = 256 нитей в блоке).
inline constexpr unsigned int kMatmulBlockDim = 16;


// Сторона квадратного тайла для кёрнела с разделяемой памятью по умолчанию.
// 16 x 16 = 256 нитей: на SM архитектуры Ampere (sm_86, до 1536 нитей) помещается 6
     блоков,
// тогда как при тайле 32 x 32 (1024 нити) — только один, что снижает загрузку SM до
     67 %.
inline constexpr unsigned int kMatmulTileDim = 16;


// Наивное умножение C = A * B без разделяемой памяти (версия из практической работы
     №2).
// Двумерная сетка: ось x — столбцы C, ось y — строки C; одна нить вычисляет один
     элемент C.
template <typename AtomT>
__global__ void kernel_matmul_naive(MatrixView<AtomT> a, MatrixView<AtomT> b,
                                    MatrixView<AtomT> c) {
    const std::size_t row = static_cast<std::size_t>(blockIdx.y) * blockDim.y +
     threadIdx.y;
    const std::size_t col = static_cast<std::size_t>(blockIdx.x) * blockDim.x +
     threadIdx.x;


    if (row < c.nrows() && col < c.ncols()) {
        AtomT acc{};
        for (std::size_t p = 0; p < a.ncols(); ++p) {
            acc += a(row, p) * b(p, col);
        }
        c(row, col) = acc;
    }
}


// Блочное умножение C = A * B с разделяемой памятью.
// Блок нитей TileDim x TileDim вычисляет тайл C того же размера. Внутренняя
     размерность k
// проходится полосами ширины TileDim: на каждом шаге нити блока совместно загружают
     тайл A
// и тайл B в разделяемую память (каждая нить — по одному элементу каждого тайла),
     после чего
// каждая нить накапливает скалярное произведение строки тайла A на столбец тайла B.
// Элементы за границами матриц заменяются нулями, поэтому размеры могут быть не
     кратны TileDim.
// Кёрнел ожидает blockDim = (TileDim, TileDim).
template <typename AtomT, unsigned int TileDim>
__global__ void kernel_matmul_shmem(MatrixView<AtomT> a, MatrixView<AtomT> b,
                                    MatrixView<AtomT> c) {
    __shared__ AtomT tile_a[TileDim][TileDim];
    __shared__ AtomT tile_b[TileDim][TileDim];


    const unsigned int tx = threadIdx.x;
    const unsigned int ty = threadIdx.y;
    const std::size_t row = static_cast<std::size_t>(blockIdx.y) * TileDim + ty;
    const std::size_t col = static_cast<std::size_t>(blockIdx.x) * TileDim + tx;


    AtomT acc{};
    for (std::size_t tile_start = 0; tile_start < a.ncols(); tile_start += TileDim) {
        // Соседние по tx нити читают соседние элементы строки — доступ объединяется.
        const std::size_t a_col = tile_start + tx;
        const std::size_t b_row = tile_start + ty;
        tile_a[ty][tx] = (row < a.nrows() && a_col < a.ncols()) ? a(row, a_col) :
     AtomT{};
        tile_b[ty][tx] = (b_row < b.nrows() && col < b.ncols()) ? b(b_row, col) :
     AtomT{};


        // Тайлы должны быть полностью загружены до начала вычислений.
        __syncthreads();
```

```
#pragma unroll
        for (unsigned int p = 0; p < TileDim; ++p) {
            // tile_a[ty][p] одинаков для всего варпа (broadcast),
            // tile_b[p][tx] — последовательные слова, конфликтов банков нет.
            acc += tile_a[ty][p] * tile_b[p][tx];
        }

        // Тайлы нельзя перезаписывать, пока все нити блока не закончат их чтение.
        __syncthreads();
    }

    // Проверка выполняется после цикла: все нити блока, включая лежащие за границами
     C,
    // должны участвовать в загрузке тайлов и доходить до __syncthreads.
    if (row < c.nrows() && col < c.ncols()) {
        c(row, col) = acc;
    }
}

// Размер сетки, покрывающей матрицу c блоками block_dim x block_dim.
template <typename AtomT>
dim3 matmul_grid(const MatrixView<AtomT>& c, unsigned int block_dim) {
    return dim3(static_cast<unsigned int>((c.ncols() + block_dim - 1) / block_dim),
                static_cast<unsigned int>((c.nrows() + block_dim - 1) / block_dim));
}

// Асинхронный запуск kernel_matmul_naive; синхронизацию выполняет вызывающая сторона.
template <typename AtomT>
void launch_matmul_naive(MatrixView<AtomT> a, MatrixView<AtomT> b, MatrixView<AtomT>
     c,
                         cudaStream_t stream = nullptr) {
    if (c.size() == 0) {
        return;  // Пустая сетка — недопустимая конфигурация запуска.
    }

    const dim3 block(kMatmulBlockDim, kMatmulBlockDim);
    kernel_matmul_naive<<<matmul_grid(c, kMatmulBlockDim), block, 0, stream>>>(a, b,
     c);
    check_cuda(cudaGetLastError(), "kernel_matmul_naive launch");
}

// Асинхронный запуск kernel_matmul_shmem; синхронизацию выполняет вызывающая сторона.
template <typename AtomT, unsigned int TileDim = kMatmulTileDim>
void launch_matmul_shmem(MatrixView<AtomT> a, MatrixView<AtomT> b, MatrixView<AtomT>
     c,
                         cudaStream_t stream = nullptr) {
    static_assert(TileDim > 0 && TileDim * TileDim <= 1024,
                  "Block of TileDim x TileDim threads exceeds the CUDA limit of
     1024");

    if (c.size() == 0) {
        return;  // Пустая сетка — недопустимая конфигурация запуска.
    }

    const dim3 block(TileDim, TileDim);
    kernel_matmul_shmem<AtomT, TileDim>
        <<<matmul_grid(c, TileDim), block, 0, stream>>>(a, b, c);
    check_cuda(cudaGetLastError(), "kernel_matmul_shmem launch");
}
```

Листинг А.7 – Файл `core/include/matmul_strategy.cuh`

```
#pragma once

#include <cuda_runtime.h>
#include <memory>

#include "kernel.cuh"
#include "matrix_view.cuh"

// Стратегия умножения матриц на устройстве (паттерн «Стратегия»).
// Реализация получает уже согласованные по размерам представления A (m x k), B (k x
     n)
// и C (m x n) и асинхронно ставит вычисление C = A * B в поток stream.
template <typename AtomT>
class MatmulStrategy {
public:
    virtual ~MatmulStrategy() = default;

    virtual void multiply(MatrixView<AtomT> a, MatrixView<AtomT> b, MatrixView<AtomT>
     c,
                          cudaStream_t stream = nullptr) const = 0;
};

// Наивный алгоритм: каждая нить читает строку A и столбец B из глобальной памяти.
template <typename AtomT>
class NaiveMatmul final : public MatmulStrategy<AtomT> {
public:
    void multiply(MatrixView<AtomT> a, MatrixView<AtomT> b, MatrixView<AtomT> c,
                  cudaStream_t stream = nullptr) const override {
        launch_matmul_naive(a, b, c, stream);
    }
};

// Блочный алгоритм с буферизацией тайлов TileDim x TileDim в разделяемой памяти.
template <typename AtomT, unsigned int TileDim = kMatmulTileDim>
class SharedMemoryMatmul final : public MatmulStrategy<AtomT> {
public:
    void multiply(MatrixView<AtomT> a, MatrixView<AtomT> b, MatrixView<AtomT> c,
                  cudaStream_t stream = nullptr) const override {
        launch_matmul_shmem<AtomT, TileDim>(a, b, c, stream);
    }
};

// Стратегия, используемая Matrix::operator* по умолчанию.
template <typename AtomT>
std::shared_ptr<const MatmulStrategy<AtomT>> default_matmul_strategy() {
    static const auto strategy = std::make_shared<const SharedMemoryMatmul<AtomT>>();
    return strategy;
}
```

Листинг А.8 – Файл `core/include/matrix.cuh`

```
#pragma once

#include <cstddef>
#include <memory>
#include <stdexcept>
#include <utility>

#include "cuda_check.cuh"
#include "data.cuh"
#include "matmul_strategy.cuh"
```

```
#include "matrix_view.cuh"

// Фасад над данными на устройстве.
// Агрегация: Data разделяется между копиями Matrix через std::shared_ptr.
// Композиция: каждый экземпляр Matrix владеет собственным MatrixView.
// Алгоритм умножения задаётся объектом-стратегией MatmulStrategy (по умолчанию —
// блочный алгоритм с разделяемой памятью); стратегия разделяется между матрицами.
template <typename AtomT>
class Matrix {
public:
    using Strategy = MatmulStrategy<AtomT>;

private:
    std::shared_ptr<Data<AtomT>> data_;
    MatrixView<AtomT> view_;
    std::shared_ptr<const Strategy> matmul_strategy_;

public:
    Matrix(std::size_t nrows, std::size_t ncols,
           std::shared_ptr<const Strategy> matmul_strategy =
     default_matmul_strategy<AtomT>())
        : data_(std::make_shared<Data<AtomT>>(nrows * ncols)),
          view_(data_->data(), nrows, ncols),
          matmul_strategy_(std::move(matmul_strategy)) {
        if (!matmul_strategy_) {
            throw std::invalid_argument("Matrix: matmul strategy must not be null");
        }
    }

    std::size_t size() const { return view_.size(); }
    std::size_t nrows() const { return view_.nrows(); }
    std::size_t ncols() const { return view_.ncols(); }

    Data<AtomT>& data() { return *data_; }
    const Data<AtomT>& data() const { return *data_; }

    MatrixView<AtomT>& view() { return view_; }
    const MatrixView<AtomT>& view() const { return view_; }

    const Strategy& matmul_strategy() const { return *matmul_strategy_; }

    void set_matmul_strategy(std::shared_ptr<const Strategy> matmul_strategy) {
        if (!matmul_strategy) {
            throw std::invalid_argument("Matrix: matmul strategy must not be null");
        }
        matmul_strategy_ = std::move(matmul_strategy);
    }

    // C = (*this) * other, где (*this) — m x k, other — k x n, C — m x n.
    // Умножение выполняет стратегия левого операнда; результат наследует её.
    Matrix operator*(const Matrix& other) const {
        if (ncols() != other.nrows()) {
            throw std::invalid_argument("Matrix dimensions mismatch: lhs.ncols !=
     rhs.nrows");
        }

        Matrix result(nrows(), other.ncols(), matmul_strategy_);
        matmul_strategy_->multiply(view_, other.view_, result.view_);
        check_cuda(cudaDeviceSynchronize(), "matmul execution");
        return result;
    }
};
```

Листинг А.9 – Файл `core/src/matrix.cu`

```
#include "matrix.cuh"


// Явные инстанцирования: проверяют компилируемость шаблонов для основных типов.
template class Data<float>;
template class Data<double>;
template class Data<int>;


template class MatrixView<float>;
template class MatrixView<double>;
template class MatrixView<int>;


template class NaiveMatmul<float>;
template class NaiveMatmul<double>;
template class NaiveMatmul<int>;


template class SharedMemoryMatmul<float>;
template class SharedMemoryMatmul<double>;
template class SharedMemoryMatmul<int>;


template class Matrix<float>;
template class Matrix<double>;
template class Matrix<int>;
```

Листинг А.10 – Файл `tests/CMakeLists.txt`

```
add_executable(unit_tests
    src/test_matmul.cu
)


target_include_directories(unit_tests PRIVATE
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)


target_compile_definitions(unit_tests PRIVATE EIGEN_NO_CUDA)


target_link_libraries(unit_tests PRIVATE
    core
    GTest::gtest_main
    Eigen3::Eigen
)


include(GoogleTest)
gtest_discover_tests(unit_tests DISCOVERY_TIMEOUT 60)
```

Листинг А.11 – Файл `tests/include/eigen_interop.cuh`

```
#pragma once


#include <Eigen/Dense>


#include "matrix.cuh"


// Eigen::MatrixXf хранит элементы по столбцам (column-major), Matrix — по строкам
     (row-major),
// поэтому обмен данными идёт через промежуточную row-major матрицу Eigen.
template <typename AtomT>
using RowMajorMatrix = Eigen::Matrix<AtomT, Eigen::Dynamic, Eigen::Dynamic,
     Eigen::RowMajor>;
```

```
template <typename AtomT>
Matrix<AtomT> to_device(const Eigen::Matrix<AtomT, Eigen::Dynamic, Eigen::Dynamic>&
     host) {
    const RowMajorMatrix<AtomT> row_major = host;
    Matrix<AtomT> device(static_cast<std::size_t>(host.rows()),
                         static_cast<std::size_t>(host.cols()));
    device.data().copy_from_host(row_major.data());
    return device;
}


template <typename AtomT>
Eigen::Matrix<AtomT, Eigen::Dynamic, Eigen::Dynamic> to_host(const Matrix<AtomT>&
     device) {
    RowMajorMatrix<AtomT> row_major(static_cast<Eigen::Index>(device.nrows()),
                                    static_cast<Eigen::Index>(device.ncols()));
    device.data().copy_to_host(row_major.data());
    return row_major;
}
```

Листинг А.12 – Файл `tests/src/test_matmul.cu`

```
#include <gtest/gtest.h>


#include <cstddef>
#include <cstdlib>
#include <memory>
#include <string>
#include <tuple>
#include <type_traits>
#include <vector>


#include "eigen_interop.cuh"
#include "matrix.cuh"


namespace {


constexpr double kPrecision = 1e-5;


const std::vector<std::size_t> kSizes = {1, 2, 3, 127, 128, 129, 512};


// Проверяемые стратегии operator*.
enum class StrategyKind { Default, Naive, Shmem16, Shmem32 };


std::shared_ptr<const MatmulStrategy<float>> make_strategy(StrategyKind kind) {
    switch (kind) {
        case StrategyKind::Naive:
            return std::make_shared<const NaiveMatmul<float>>();
        case StrategyKind::Shmem16:
            return std::make_shared<const SharedMemoryMatmul<float, 16>>();
        case StrategyKind::Shmem32:
            return std::make_shared<const SharedMemoryMatmul<float, 32>>();
        case StrategyKind::Default:
        default:
            return default_matmul_strategy<float>();
    }
}


std::string to_string(StrategyKind kind) {
    switch (kind) {
        case StrategyKind::Naive:
            return "Naive";
        case StrategyKind::Shmem16:
```

```
            return "Shmem16";
        case StrategyKind::Shmem32:
            return "Shmem32";
        case StrategyKind::Default:
        default:
            return "Default";
    }
}

}  // namespace

// ---------------------------------------------------------------------------
// Параметризованные тесты: A (m x k) * B (k x n) для всех m, n, k из kSizes.
// ---------------------------------------------------------------------------

using MatmulParams = std::tuple<std::size_t, std::size_t, std::size_t>;  // m, k, n

class MatmulParamTest : public ::testing::TestWithParam<MatmulParams> {};

// Основной набор: operator* со стратегией по умолчанию (kernel_matmul_shmem).
TEST_P(MatmulParamTest, MatchesEigen) {
    const auto [m, k, n] = GetParam();

    std::srand(static_cast<unsigned int>(m * 1'000'003 + k * 1'009 + n));
    const Eigen::MatrixXf a_host = Eigen::MatrixXf::Random(m, k);
    const Eigen::MatrixXf b_host = Eigen::MatrixXf::Random(k, n);
    const Eigen::MatrixXf expected = a_host * b_host;

    const Matrix<float> a = to_device(a_host);
    const Matrix<float> b = to_device(b_host);
    const Matrix<float> c = a * b;

    ASSERT_EQ(c.nrows(), m);
    ASSERT_EQ(c.ncols(), n);
    ASSERT_EQ(c.size(), m * n);

    const Eigen::MatrixXf actual = to_host(c);
    EXPECT_TRUE(actual.isApprox(expected, kPrecision))
        << "max |diff| = " << (actual - expected).cwiseAbs().maxCoeff();
}

std::string matmul_param_name(const ::testing::TestParamInfo<MatmulParams>& info) {
    const auto [m, k, n] = info.param;
    return "m" + std::to_string(m) + "_k" + std::to_string(k) + "_n" +
     std::to_string(n);
}

INSTANTIATE_TEST_SUITE_P(AllSizes, MatmulParamTest,
                         ::testing::Combine(::testing::ValuesIn(kSizes),
                                            ::testing::ValuesIn(kSizes),
                                            ::testing::ValuesIn(kSizes)),
                         matmul_param_name);

// ---------------------------------------------------------------------------
// Те же размеры для каждой стратегии в отдельности (в т. ч. предыдущей версии и
// блочного алгоритма с тайлами 16 x 16 и 32 x 32).
// ---------------------------------------------------------------------------

using StrategyParams = std::tuple<StrategyKind, std::size_t, std::size_t,
     std::size_t>;

class MatmulStrategyParamTest : public ::testing::TestWithParam<StrategyParams> {};
```

```
TEST_P(MatmulStrategyParamTest, MatchesEigen) {
    const auto [kind, m, k, n] = GetParam();


    std::srand(static_cast<unsigned int>(m * 1'000'003 + k * 1'009 + n));
    const Eigen::MatrixXf a_host = Eigen::MatrixXf::Random(m, k);
    const Eigen::MatrixXf b_host = Eigen::MatrixXf::Random(k, n);
    const Eigen::MatrixXf expected = a_host * b_host;


    Matrix<float> a = to_device(a_host);
    const Matrix<float> b = to_device(b_host);
    a.set_matmul_strategy(make_strategy(kind));
    const Matrix<float> c = a * b;


    ASSERT_EQ(c.nrows(), m);
    ASSERT_EQ(c.ncols(), n);


    const Eigen::MatrixXf actual = to_host(c);
    EXPECT_TRUE(actual.isApprox(expected, kPrecision))
        << "max |diff| = " << (actual - expected).cwiseAbs().maxCoeff();
}


std::string strategy_param_name(const ::testing::TestParamInfo<StrategyParams>& info)
     {
    const auto [kind, m, k, n] = info.param;
    return to_string(kind) + "_m" + std::to_string(m) + "_k" + std::to_string(k) +
     "_n" +
          std::to_string(n);
}


INSTANTIATE_TEST_SUITE_P(AllStrategies, MatmulStrategyParamTest,
                         ::testing::Combine(::testing::Values(StrategyKind::Naive,
                                                              StrategyKind::Shmem16,
                                                              StrategyKind::Shmem32),
                                            ::testing::ValuesIn(kSizes),
                                            ::testing::ValuesIn(kSizes),
                                            ::testing::ValuesIn(kSizes)),
                         strategy_param_name);


// ----------------------------------------------------------------------------
// Граничные случаи.
// ----------------------------------------------------------------------------


TEST(MatmulTest, KnownSmallProduct) {
    // [1 2 3]   [ 7  8]   [ 58  64]
    // [4 5 6] * [ 9 10] = [139 154]
    //           [11 12]
    const std::vector<float> a_host = {1, 2, 3, 4, 5, 6};
    const std::vector<float> b_host = {7, 8, 9, 10, 11, 12};
    Matrix<float> a(2, 3);
    Matrix<float> b(3, 2);
    a.data().copy_from_host(a_host.data());
    b.data().copy_from_host(b_host.data());


    const Matrix<float> c = a * b;


    std::vector<float> c_host(c.size());
    c.data().copy_to_host(c_host.data());
    EXPECT_EQ(c_host, (std::vector<float>{58, 64, 139, 154}));
}


TEST(MatmulTest, IdentityIsNeutral) {
    constexpr std::size_t n = 129;
```

```
    std::srand(7);
    const Eigen::MatrixXf a_host = Eigen::MatrixXf::Random(n, n);

    const Matrix<float> a = to_device(a_host);
    const Matrix<float> identity = to_device<float>(Eigen::MatrixXf::Identity(n, n));

    EXPECT_TRUE(to_host(a * identity).isApprox(a_host, kPrecision));
    EXPECT_TRUE(to_host(identity * a).isApprox(a_host, kPrecision));
}

// Вектор-строка на вектор-столбец и наоборот: сетка из одного блока и сетка
// с вырожденной по одной оси формой; внутренняя размерность пересекает много тайлов.
TEST(MatmulTest, VectorShapes) {
    std::srand(17);
    const Eigen::MatrixXf row = Eigen::MatrixXf::Random(1, 1000);
    const Eigen::MatrixXf col = Eigen::MatrixXf::Random(1000, 1);

    EXPECT_TRUE(to_host(to_device(row) * to_device(col)).isApprox(row * col,
     kPrecision));
    EXPECT_TRUE(to_host(to_device(col) * to_device(row)).isApprox(col * row,
     kPrecision));
}

// Размеры вокруг границ тайлов 16 и 32: проверяют загрузку неполных тайлов нулями.
TEST(MatmulTest, TileBoundaries) {
    const std::vector<std::size_t> sizes = {15, 16, 17, 31, 32, 33, 63, 65};
    for (const std::size_t n : sizes) {
        std::srand(static_cast<unsigned int>(n));
        const Eigen::MatrixXf a_host = Eigen::MatrixXf::Random(n, n + 1);
        const Eigen::MatrixXf b_host = Eigen::MatrixXf::Random(n + 1, n - 1);

        const Eigen::MatrixXf actual = to_host(to_device(a_host) * to_device(b_host));
        EXPECT_TRUE(actual.isApprox(a_host * b_host, kPrecision)) << "n = " << n;
    }
}

// Результат не зависит от количества запусков: повторное умножение не должно
// использовать данные, оставшиеся в разделяемой памяти от предыдущих запусков.
TEST(MatmulTest, RepeatedProductIsStable) {
    std::srand(19);
    const Eigen::MatrixXf a_host = Eigen::MatrixXf::Random(100, 70);
    const Eigen::MatrixXf b_host = Eigen::MatrixXf::Random(70, 90);
    const Matrix<float> a = to_device(a_host);
    const Matrix<float> b = to_device(b_host);

    const Eigen::MatrixXf first = to_host(a * b);
    const Eigen::MatrixXf second = to_host(a * b);

    EXPECT_EQ(first, second);
}

TEST(MatmulTest, OperandsAreNotModified) {
    std::srand(11);
    const Eigen::MatrixXf a_host = Eigen::MatrixXf::Random(64, 32);
    const Eigen::MatrixXf b_host = Eigen::MatrixXf::Random(32, 48);
    const Matrix<float> a = to_device(a_host);
    const Matrix<float> b = to_device(b_host);

    const Matrix<float> c = a * b;

    EXPECT_EQ(to_host(a), a_host);
    EXPECT_EQ(to_host(b), b_host);
    EXPECT_NE(c.data().data(), a.data().data());
```

```
    EXPECT_NE(c.data().data(), b.data().data());
}


TEST(MatmulTest, DimensionMismatchThrows) {
    const Matrix<float> a(3, 4);
    const Matrix<float> b(5, 3);
    EXPECT_THROW(a * b, std::invalid_argument);
}


TEST(MatmulTest, ZeroInnerDimensionGivesZeroMatrix) {
    const Matrix<float> a(4, 0);
    const Matrix<float> b(0, 5);


    const Matrix<float> c = a * b;


    ASSERT_EQ(c.nrows(), 4u);
    ASSERT_EQ(c.ncols(), 5u);
    EXPECT_TRUE(to_host(c).isZero());
}


TEST(MatmulTest, EmptyResult) {
    const Matrix<float> a(0, 3);
    const Matrix<float> b(3, 2);


    const Matrix<float> c = a * b;


    EXPECT_EQ(c.nrows(), 0u);
    EXPECT_EQ(c.ncols(), 2u);
    EXPECT_EQ(c.size(), 0u);
}


TEST(MatmulTest, DoublePrecision) {
    std::srand(13);
    const Eigen::MatrixXd a_host = Eigen::MatrixXd::Random(129, 127);
    const Eigen::MatrixXd b_host = Eigen::MatrixXd::Random(127, 3);


    const Matrix<double> c = to_device(a_host) * to_device(b_host);


    EXPECT_TRUE(to_host(c).isApprox(a_host * b_host, 1e-12));
}


TEST(MatmulTest, IntegerProductIsExact) {
    Eigen::Matrix<int, Eigen::Dynamic, Eigen::Dynamic> a_host(3, 2);
    Eigen::Matrix<int, Eigen::Dynamic, Eigen::Dynamic> b_host(2, 4);
    a_host << 1, -2, 3, 4, -5, 6;
    b_host << 1, 2, 3, 4, 5, 6, 7, 8;


    const Matrix<int> c = to_device(a_host) * to_device(b_host);


    EXPECT_EQ(to_host(c), (a_host * b_host).eval());
}


// Многократное умножение не должно приводить к утечке памяти устройства:
// после уничтожения всех матриц объём свободной памяти возвращается к исходному.
TEST(MatmulTest, NoDeviceMemoryLeak) {
    constexpr std::size_t n = 512;
    constexpr int kIterations = 50;


    // Первое обращение к CUDA и умножение инициализируют контекст и модуль заранее.
    { const Matrix<float> warm_up = Matrix<float>(n, n) * Matrix<float>(n, n); }
```

```
    std::size_t free_before = 0;
    std::size_t total = 0;
    check_cuda(cudaMemGetInfo(&free_before, &total), "cudaMemGetInfo");


    {
        const Matrix<float> a(n, n);
        const Matrix<float> b(n, n);
        for (int i = 0; i < kIterations; ++i) {
            const Matrix<float> c = a * b;
            const Matrix<float> copy = c;  // разделяет данные, не выделяет память
            Data<float> deep_copy(c.data());
        }
    }


    std::size_t free_after = 0;
    check_cuda(cudaMemGetInfo(&free_after, &total), "cudaMemGetInfo");
    EXPECT_EQ(free_after, free_before);
}

// ----------------------------------------------------------------------------
// Стратегии умножения.
// ----------------------------------------------------------------------------


TEST(MatmulStrategyTest, DefaultIsSharedMemory) {
    const Matrix<float> a(2, 2);
    EXPECT_NE(dynamic_cast<const SharedMemoryMatmul<float>*>(&a.matmul_strategy()),
     nullptr);
}


TEST(MatmulStrategyTest, ResultInheritsLhsStrategy) {
    const auto naive = std::make_shared<const NaiveMatmul<float>>();
    const Matrix<float> a(3, 4, naive);
    const Matrix<float> b(4, 5);


    const Matrix<float> c = a * b;


    EXPECT_EQ(&c.matmul_strategy(), naive.get());
}


TEST(MatmulStrategyTest, NullStrategyThrows) {
    EXPECT_THROW(Matrix<float>(2, 2, nullptr), std::invalid_argument);


    Matrix<float> a(2, 2);
    EXPECT_THROW(a.set_matmul_strategy(nullptr), std::invalid_argument);
}


TEST(MatmulStrategyTest, StrategiesAgree) {
    std::srand(23);
    const Eigen::MatrixXf a_host = Eigen::MatrixXf::Random(300, 200);
    const Eigen::MatrixXf b_host = Eigen::MatrixXf::Random(200, 100);
    Matrix<float> a = to_device(a_host);
    const Matrix<float> b = to_device(b_host);


    a.set_matmul_strategy(make_strategy(StrategyKind::Naive));
    const Eigen::MatrixXf naive = to_host(a * b);
    a.set_matmul_strategy(make_strategy(StrategyKind::Default));
    const Eigen::MatrixXf shmem = to_host(a * b);


    EXPECT_TRUE(shmem.isApprox(naive, kPrecision));
}


// ----------------------------------------------------------------------------
```

```
// Matrix / MatrixView / Data.
// ----------------------------------------------------------------------------

TEST(MatrixViewTest, IsTriviallyCopyable) {
    static_assert(std::is_trivially_copyable_v<MatrixView<float>>);
    static_assert(std::is_trivially_copyable_v<MatrixView<double>>);
    static_assert(std::is_trivially_copyable_v<MatrixView<int>>);
}

TEST(MatrixViewTest, RowMajorIndexing) {
    std::vector<int> storage(12);
    MatrixView<int> view(storage.data(), 3, 4);
    for (std::size_t i = 0; i < view.nrows(); ++i) {
        for (std::size_t j = 0; j < view.ncols(); ++j) {
            view(i, j) = static_cast<int>(i * 10 + j);
        }
    }

    EXPECT_EQ(view.size(), 12u);
    EXPECT_EQ(view[0], 0);
    EXPECT_EQ(view[3], 3);
    EXPECT_EQ(view[4], 10);
    EXPECT_EQ(view[11], 23);
}

TEST(MatrixTest, CopiesShareData) {
    Matrix<float> a(2, 2);
    const Matrix<float> b = a;  // NOLINT: проверяем именно копирование

    EXPECT_EQ(a.data().data(), b.data().data());
    EXPECT_NE(&a.view(), &b.view());  // собственное представление у каждого
     экземпляра
    EXPECT_EQ(&a.matmul_strategy(), &b.matmul_strategy());

    const std::vector<float> values = {1, 2, 3, 4};
    a.data().copy_from_host(values.data());
    std::vector<float> read_back(4);
    b.data().copy_to_host(read_back.data());
    EXPECT_EQ(read_back, values);
}

TEST(MatrixTest, DimensionsAndSize) {
    const Matrix<float> a(3, 7);
    EXPECT_EQ(a.nrows(), 3u);
    EXPECT_EQ(a.ncols(), 7u);
    EXPECT_EQ(a.size(), 21u);
    EXPECT_EQ(a.data().size(), 21u);
}

TEST(DataTest, CopyIsDeepAndMoveTransfersOwnership) {
    const std::vector<float> values = {1, 2, 3};
    Data<float> original(3);
    original.copy_from_host(values.data());

    Data<float> copy(original);
    EXPECT_NE(copy.data(), original.data());
    std::vector<float> read_back(3);
    copy.copy_to_host(read_back.data());
    EXPECT_EQ(read_back, values);

    const float* raw = copy.data();
    Data<float> moved(std::move(copy));
    EXPECT_EQ(moved.data(), raw);
    EXPECT_EQ(copy.data(), nullptr);  // NOLINT: проверяем состояние после перемещения
```

```
    EXPECT_EQ(copy.size(), 0u);
}
```

Листинг А.13 – Файл `benchmarks/CMakeLists.txt`

```
add_executable(benchmarks
    src/bench_matmul.cu
)

target_include_directories(benchmarks PRIVATE
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)

target_compile_definitions(benchmarks PRIVATE EIGEN_NO_CUDA)

target_link_libraries(benchmarks PRIVATE
    core
    benchmark::benchmark
    Eigen3::Eigen
)
```

Листинг А.14 – Файл `benchmarks/include/cuda_event_timer.cuh`

```
#pragma once

#include <cuda_runtime.h>

#include "cuda_check.cuh"

// RAII-обёртка над парой CUDA Events для измерения времени работы кода на устройстве.
class CudaEventTimer {
private:
    cudaEvent_t start_{};
    cudaEvent_t stop_{};

public:
    CudaEventTimer() {
        check_cuda(cudaEventCreate(&start_), "cudaEventCreate");
        const cudaError_t err = cudaEventCreate(&stop_);
        if (err != cudaSuccess) {
            cudaEventDestroy(start_);
            check_cuda(err, "cudaEventCreate");
        }
    }

    ~CudaEventTimer() {
        cudaEventDestroy(start_);
        cudaEventDestroy(stop_);
    }

    CudaEventTimer(const CudaEventTimer&) = delete;
    CudaEventTimer& operator=(const CudaEventTimer&) = delete;

    void start(cudaStream_t stream = nullptr) {
        check_cuda(cudaEventRecord(start_, stream), "cudaEventRecord(start)");
    }

    // Возвращает время между start() и stop() в миллисекундах.
    float stop(cudaStream_t stream = nullptr) {
        check_cuda(cudaEventRecord(stop_, stream), "cudaEventRecord(stop)");
        check_cuda(cudaEventSynchronize(stop_), "cudaEventSynchronize");
        float elapsed_ms = 0.0f;
```

```
        check_cuda(cudaEventElapsedTime(&elapsed_ms, start_, stop_),
     "cudaEventElapsedTime");
        return elapsed_ms;
    }
};
```

Листинг А.15 – Файл `benchmarks/src/bench_matmul.cu`

```
#include <benchmark/benchmark.h>


#include <Eigen/Dense>
#include <cstddef>
#include <cstdint>
#include <cstdlib>
#include <string>


#include "cuda_event_timer.cuh"
#include "matrix.cuh"


namespace {


constexpr std::int64_t kMinSize = 16;
constexpr std::int64_t kMaxSize = 1024;
constexpr int kRepetitions = 10;


// Заполняет матрицу на устройстве случайными значениями из [-1, 1].
void fill_random(Matrix<float>& matrix) {
    using RowMajorXf = Eigen::Matrix<float, Eigen::Dynamic, Eigen::Dynamic,
     Eigen::RowMajor>;
    const RowMajorXf host =
     RowMajorXf::Random(static_cast<Eigen::Index>(matrix.nrows()),

     static_cast<Eigen::Index>(matrix.ncols()));
    matrix.data().copy_from_host(host.data());
}


// Параметры устройства попадают в поле context JSON-отчёта.
void add_device_context() {
    int device = 0;
    check_cuda(cudaGetDevice(&device), "cudaGetDevice");
    cudaDeviceProp prop{};
    check_cuda(cudaGetDeviceProperties(&prop, device), "cudaGetDeviceProperties");


    int clock_khz = 0;
    int memory_clock_khz = 0;
    check_cuda(cudaDeviceGetAttribute(&clock_khz, cudaDevAttrClockRate, device),
               "cudaDeviceGetAttribute(ClockRate)");
    check_cuda(cudaDeviceGetAttribute(&memory_clock_khz, cudaDevAttrMemoryClockRate,
     device),
               "cudaDeviceGetAttribute(MemoryClockRate)");


    benchmark::AddCustomContext("gpu_name", prop.name);
    benchmark::AddCustomContext("gpu_compute_capability",
                                std::to_string(prop.major) + "." +
     std::to_string(prop.minor));
    benchmark::AddCustomContext("gpu_sm_count",
     std::to_string(prop.multiProcessorCount));
    benchmark::AddCustomContext("gpu_clock_mhz", std::to_string(clock_khz / 1000));
    benchmark::AddCustomContext("gpu_memory_clock_mhz",
     std::to_string(memory_clock_khz / 1000));
    benchmark::AddCustomContext("gpu_memory_bus_width_bit",
                                std::to_string(prop.memoryBusWidth));
    benchmark::AddCustomContext("gpu_l2_cache_bytes",
     std::to_string(prop.l2CacheSize));
```

```
    benchmark::AddCustomContext("gpu_shared_mem_per_block_bytes",
                                std::to_string(prop.sharedMemPerBlock));
}


}  // namespace


// Matrix::operator* = выделение памяти под результат + strategy.multiply +
     синхронизация.
// Выделение/копирование/освобождение памяти вынесены за пределы цикла, а внутри цикла
// CUDA Events измеряют только время выполнения кёрнела выбранной стратегии на
     устройстве.
template <typename Strategy>
static void BM_Matrix_operator_mul(benchmark::State& state) {
    const auto n = static_cast<std::size_t>(state.range(0));


    std::srand(42);
    Matrix<float> a(n, n);
    Matrix<float> b(n, n);
    Matrix<float> c(n, n);
    fill_random(a);
    fill_random(b);


    const Strategy strategy{};
    CudaEventTimer timer;


    // Прогрев: загрузка модуля, выход GPU из режима энергосбережения.
    strategy.multiply(a.view(), b.view(), c.view());
    check_cuda(cudaDeviceSynchronize(), "warm-up");


    for (auto _ : state) {
        timer.start();
        strategy.multiply(a.view(), b.view(), c.view());
        const float elapsed_ms = timer.stop();
        state.SetIterationTime(static_cast<double>(elapsed_ms) * 1e-3);
    }


    state.counters["n"] = static_cast<double>(n);
    state.counters["flop"] = 2.0 * static_cast<double>(n) * n * n;
}


#define MATMUL_BENCHMARK(strategy)                                          \
    BENCHMARK_TEMPLATE(BM_Matrix_operator_mul, strategy)                    \
        ->RangeMultiplier(2)                                                \
        ->Range(kMinSize, kMaxSize)                                         \
        ->UseManualTime()                                                   \
        ->Repetitions(kRepetitions)                                         \
        ->ReportAggregatesOnly(true)                                        \
        ->Unit(benchmark::kMicrosecond)


// Предыдущая версия operator* (практическая работа №2).
using Naive = NaiveMatmul<float>;
// Новая версия operator*: тайлы 16 x 16 (по умолчанию) и 32 x 32.
using Shmem16 = SharedMemoryMatmul<float, 16>;
using Shmem32 = SharedMemoryMatmul<float, 32>;


MATMUL_BENCHMARK(Naive);
MATMUL_BENCHMARK(Shmem16);
MATMUL_BENCHMARK(Shmem32);


int main(int argc, char** argv) {
    benchmark::Initialize(&argc, argv);
    if (benchmark::ReportUnrecognizedArguments(argc, argv)) {
        return 1;
    }
```

```
    add_device_context();
    benchmark::RunSpecifiedBenchmarks();
    benchmark::Shutdown();
    return 0;
}
```

Листинг А.16 – Файл `scripts/bench_data.py`

```
"""Загрузка результатов Google Benchmark (JSON) для построения графиков."""

import json
from dataclasses import dataclass
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
DEFAULT_JSON = ROOT / "results" / "benchmark_results.json"
IMAGES_DIR = ROOT / "docs" / "images"

NAIVE = "BM_Matrix_operator_mul<Naive>"
SHMEM16 = "BM_Matrix_operator_mul<Shmem16>"
SHMEM32 = "BM_Matrix_operator_mul<Shmem32>"

LABELS = {
    NAIVE: "kernel_matmul_naive (работа №2)",
    SHMEM16: "kernel_matmul_shmem, тайл 16×16",
    SHMEM32: "kernel_matmul_shmem, тайл 32×32",
}

COLORS = {NAIVE: "#d95f02", SHMEM16: "#1b9e77", SHMEM32: "#7570b3"}
SYMBOLS = {NAIVE: "circle", SHMEM16: "square", SHMEM32: "diamond"}

# Множители перевода в секунды для поля time_unit.
_TO_SECONDS = {"ns": 1e-9, "us": 1e-6, "ms": 1e-3, "s": 1.0}


@dataclass
class Series:
    sizes: list[int]
    median_s: list[float]
    stddev_s: list[float]


def load_json(json_path: Path = DEFAULT_JSON) -> dict:
    # На Windows путь к исполняемому файлу в "context" пишется в ANSI-кодировке.
    with open(json_path, encoding="utf-8", errors="replace") as f:
        return json.load(f)


def load(json_path: Path = DEFAULT_JSON) -> dict[str, Series]:
    """Возвращает {имя бенчмарка: Series} по агрегатам median и stddev."""
    stats: dict[str, dict[int, dict[str, float]]] = {}
    for bench in load_json(json_path)["benchmarks"]:
        aggregate = bench.get("aggregate_name")
        if aggregate not in ("median", "stddev"):
            continue
        family, size = bench["run_name"].split("/")[:2]
        seconds = bench["real_time"] * _TO_SECONDS[bench["time_unit"]]
        stats.setdefault(family, {}).setdefault(int(size), {})[aggregate] = seconds

    result = {}
    for family, by_size in stats.items():
```

```
        sizes = sorted(by_size)
        result[family] = Series(
            sizes=sizes,
            median_s=[by_size[n]["median"] for n in sizes],
            stddev_s=[by_size[n].get("stddev", 0.0) for n in sizes],
        )
    return result



def gflops(n: int, seconds: float) -> float:
    """Производительность умножения n x n матриц: 2 n^3 операций."""
    return 2.0 * n**3 / seconds / 1e9



def save(fig, name: str) -> None:
    """Сохраняет график в docs/images в форматах PNG и HTML (интерактивный)."""
    IMAGES_DIR.mkdir(parents=True, exist_ok=True)
    fig.write_html(IMAGES_DIR / f"{name}.html", include_plotlyjs="cdn")
    try:
        fig.write_image(IMAGES_DIR / f"{name}.png", width=1000, height=600, scale=2)
        print(f"Saved {IMAGES_DIR / name}.png / .html")
    except Exception as exc:  # kaleido требует установленный Chrome
        print(f"PNG export failed ({exc}); saved {IMAGES_DIR / name}.html only")
```

Листинг А.17 – Файл `scripts/complexity_chart.py`

```
"""График реальной вычислительной сложности T(n) для умножения n x n матриц.

Запуск из корня проекта: python scripts/complexity_chart.py
     [path/to/benchmark_results.json]
"""

import sys
from pathlib import Path

import numpy as np
import plotly.graph_objects as go

from bench_data import (COLORS, DEFAULT_JSON, LABELS, NAIVE, SHMEM16, SHMEM32,
     SYMBOLS,
                        gflops, load, save)

FAMILIES = (NAIVE, SHMEM16, SHMEM32)



def fit_exponent(sizes: list[int], times: list[float], min_size: int) -> float:
    """Показатель степени p в T(n) ~ c * n^p (МНК в логарифмических координатах)."""
    pts = [(n, t) for n, t in zip(sizes, times) if n >= min_size]
    log_n, log_t = np.log([p[0] for p in pts]), np.log([p[1] for p in pts])
    return float(np.polyfit(log_n, log_t, 1)[0])



def main() -> None:
    json_path = Path(sys.argv[1]) if len(sys.argv) > 1 else DEFAULT_JSON
    series = load(json_path)


    fig = go.Figure()
    for family in FAMILIES:
        s = series[family]
        fig.add_trace(go.Scatter(
            x=s.sizes, y=[t * 1e3 for t in s.median_s],
```

```
            error_y=dict(type="data", array=[d * 1e3 for d in s.stddev_s],
     visible=True),
            mode="lines+markers", name=LABELS[family],
            line=dict(color=COLORS[family], width=2),
            marker=dict(symbol=SYMBOLS[family], size=9),
        ))

    # Теоретическая оценка O(n^3), привязанная к последней точке измерений.
    for family in (NAIVE, SHMEM16):
        s = series[family]
        n_max, t_max = s.sizes[-1], s.median_s[-1]
        fig.add_trace(go.Scatter(
            x=s.sizes, y=[t_max * (n / n_max) ** 3 * 1e3 for n in s.sizes],
            mode="lines", name=f"c·n³ ({LABELS[family].split(',')[0].split(' ')[0]})",
            line=dict(color=COLORS[family], width=1, dash="dash"),
        ))

    fig.update_layout(
        title="Реальная вычислительная сложность умножения матриц n×n",
        xaxis=dict(title="Размер матрицы n", type="log",
     tickvals=series[NAIVE].sizes),
        yaxis=dict(title="Время T(n), мс", type="log", exponentformat="power"),
        template="plotly_white", legend=dict(x=0.01, y=0.99),
    )
    save(fig, "complexity_chart")

    # Таблица для отчёта.
    header = f"{'n':>6}" + "".join(f" | {name.split('<')[1][:-1]:>9}, мкс | GFLOP/s"
                                   for name in FAMILIES)
    print("\n" + header)
    for i, n in enumerate(series[NAIVE].sizes):
        row = f"{n:>6}"
        for family in FAMILIES:
            t = series[family].median_s[i]
            row += f" | {t * 1e6:>14.1f} | {gflops(n, t):>7.1f}"
        print(row)

    for family in FAMILIES:
        s = series[family]
        print(f"{family}: p(n >= 16) = {fit_exponent(s.sizes, s.median_s, 16):.2f}, "
              f"p(n >= 256) = {fit_exponent(s.sizes, s.median_s, 256):.2f}")


if __name__ == "__main__":
    main()
```

Листинг А.18 – Файл `scripts/speedup_chart.py`

```
"""График ускорения S(n) = T_naive(n) / T_shmem(n) новой реализации operator*.

Запуск из корня проекта: python scripts/speedup_chart.py
     [path/to/benchmark_results.json]
"""

import sys
from pathlib import Path

import plotly.graph_objects as go

from bench_data import COLORS, DEFAULT_JSON, LABELS, NAIVE, SHMEM16, SHMEM32, SYMBOLS,
     load, save
```

```
def main() -> None:
    json_path = Path(sys.argv[1]) if len(sys.argv) > 1 else DEFAULT_JSON
    series = load(json_path)


    naive = dict(zip(series[NAIVE].sizes, series[NAIVE].median_s))


    fig = go.Figure()
    for family in (SHMEM16, SHMEM32):
        shmem = dict(zip(series[family].sizes, series[family].median_s))
        sizes = sorted(naive.keys() & shmem.keys())
        speedup = [naive[n] / shmem[n] for n in sizes]


        fig.add_trace(go.Scatter(
            x=sizes, y=speedup, mode="lines+markers+text", name=LABELS[family],
            text=[f"{s:.2f}×" for s in speedup],
            textposition="top left" if family == SHMEM16 else "bottom right",
            line=dict(color=COLORS[family], width=2),
            marker=dict(symbol=SYMBOLS[family], size=9),
        ))


        print(LABELS[family])
        for n, s in zip(sizes, speedup):
            print(f"  n = {n:>5}: S = {s:6.2f}")


    fig.add_hline(y=1.0, line=dict(color="red", dash="dash"))  # граница S = 1


    fig.update_layout(
        title="Ускорение kernel_matmul_shmem относительно kernel_matmul_naive",
        xaxis=dict(title="Размер матрицы n", type="log",
     tickvals=series[NAIVE].sizes),
        yaxis=dict(title="Ускорение S(n) = T_naive / T_shmem, раз"),
        template="plotly_white", legend=dict(x=0.01, y=0.99),
    )
    save(fig, "speedup_chart")



if __name__ == "__main__":
    main()
```

Листинг А.19 – Файл `scripts/performance_chart.py`

```
"""График производительности P(n) = 2n^3 / T(n) с теоретическими верхними оценками.


Оценки строятся по параметрам GPU из поля context JSON-отчёта (число SM, частота):
  * пиковая производительность FP32: 128 FMA/такт на SM (Ampere, sm_86);
  * предел kernel_matmul_naive: на одну FMA варп выполняет 2 загрузки из кэша L1
    (строка A — broadcast, столбец B — объединённая), L1 выдаёт 128 байт/такт на SM,
    т.е. одну загрузку варпа за такт -> 0.5 варп-FMA/такт;
  * предел kernel_matmul_shmem (тайл 16): на 16 FMA варп выполняет 16 LDS (тайл B)
    и 4 LDS.128 (тайл A, broadcast) -> 16/20 = 0.8 варп-FMA/такт.


Запуск из корня проекта: python scripts/performance_chart.py
     [path/to/benchmark_results.json]
"""


import sys
from pathlib import Path


import plotly.graph_objects as go
```

```
from bench_data import (COLORS, DEFAULT_JSON, LABELS, NAIVE, SHMEM16, SHMEM32,
     SYMBOLS,
                        gflops, load, load_json, save)

WARP_SIZE = 32
FP32_LANES_PER_SM = 128



def bounds_gflops(context: dict) -> dict[str, float]:
    sm_count = int(context["gpu_sm_count"])
    clock_ghz = int(context["gpu_clock_mhz"]) / 1e3


    def per_warp_fma_rate(warp_fma_per_clock: float) -> float:
        return 2.0 * warp_fma_per_clock * WARP_SIZE * sm_count * clock_ghz


    return {
        "peak": 2.0 * FP32_LANES_PER_SM * sm_count * clock_ghz,
        NAIVE: per_warp_fma_rate(1 / 2),
        SHMEM16: per_warp_fma_rate(16 / 20),
    }



def main() -> None:
    json_path = Path(sys.argv[1]) if len(sys.argv) > 1 else DEFAULT_JSON
    series = load(json_path)
    bounds = bounds_gflops(load_json(json_path)["context"])


    fig = go.Figure()
    for family in (NAIVE, SHMEM16, SHMEM32):
        s = series[family]
        fig.add_trace(go.Scatter(
            x=s.sizes, y=[gflops(n, t) for n, t in zip(s.sizes, s.median_s)],
            mode="lines+markers", name=LABELS[family],
            line=dict(color=COLORS[family], width=2),
            marker=dict(symbol=SYMBOLS[family], size=9),
        ))


    for family, text in ((NAIVE, "предел L1 для naive"),
                         (SHMEM16, "предел shared memory для shmem 16×16")):
        fig.add_hline(y=bounds[family], line=dict(color=COLORS[family], dash="dash",
     width=1),
                      annotation_text=f"{text}: {bounds[family]:.0f} GFLOP/s",
                      annotation_position="top left")


    fig.update_layout(
        title="Производительность умножения матриц n×n "
              f"(пиковая FP32 ≈ {bounds['peak']:.0f} GFLOP/s)",
        xaxis=dict(title="Размер матрицы n", type="log",
     tickvals=series[NAIVE].sizes),
        yaxis=dict(title="Производительность P(n), GFLOP/s",
                   range=[0, bounds[SHMEM16] * 1.12]),
        template="plotly_white", legend=dict(x=0.01, y=0.75),
    )
    save(fig, "performance_chart")


    for name, value in bounds.items():
        print(f"{name}: {value:.0f} GFLOP/s")



if __name__ == "__main__":
    main()
```